

Project Report:

➤ The main logic behind your idea:

The main logic of the program will play a digital clock, maintaining time in 12-hour increments (hh: mm). The user is asked to enter any initial time in hours, minutes, and seconds; this information is kept in store until the time is constantly displayed. The clock counts seconds and maintains any overflow by carrying over into minutes and hours whenever necessary. At the instance when the time of the clock shows the original inputted time, the clock should stop and display a "Clock Stopped" notification.

The program operates in the following manner:

The user is asked to input the current seconds, minutes, and hours of a time between 0-9. The input value is stored in the Current and Initial time variables for comparison at later stages

- **Time Display:** The clock displays the time continuously available as follows hh: mm: ss.
- **Time Update:** In each cycle, the system increases the time and handles overflow from seconds to minutes and minute to hours. After reaching 12:00:00, it goes back to 00:00:00.
- **Stop Condition:** The program compares the current time with the initial time set, and whenever the current time coincides with the initial time then the clock stops.

➤ Any major assumption that you have made:

- **Valid User Input:** Application assumes users will input a time between 0 to 9. Non-numeric inputs and out-of-range inputs are not handled.
- **12 Hour Format:** Since the clock is reset after 12:00:00, the application believes that the time will be incremented in this cycle without considering the AM/PM.
- **No Time Zone or Daylight-Saving Time Modifications:** The program assumes that no modification is required because of time zone or daylight-saving time. It just disowns some practical considerations and carries out the incrementation of time in a pre-defined fashion.

➤ Limitations:

- **Limited Input Range for Time:** The program will accept input values only within a limited range, as it accepts single-digit values of hours, minutes, and seconds, which range from 0 to 9. This may create a problem in understanding and may create some sort of misunderstanding, like inserting "10" minutes or "12" hours.
- **No Error Check:** Mistakes or wrong entry is not checked. The program may behave erratically or may crash if the user happens to enter a non-numeric character or values greater than that intended.
- **Fixed 12-Hour System:** The clock resets at 12:00:00 and it's based on a 12-hour system. A 24-hour time system along with the AM to PM changeover would need to be sorted.

➤ Technical difficulty that you have faced:

- **Overflow of Time:** Ensuring the clock advances and is reset at appropriate times required close monitoring during complex processes that governed transitions from seconds to minutes, from minutes to hours, and beyond 12 hours.
- **Time Display Stack Manipulation:** Technically, the most challenging task was the manipulation of the stack used to store and retrieve the time elements momentarily while the display was made. The proper prioritization in maintaining the stack was quite essential to avoid corruption.
- **Stop Condition Management:** It was a little challenging to design the method for checking if current time equaled the initial time, considering multiple cycles of time increases. The program needed to make sure that it did the time comparison without any mistake and also did not introduce some form of bugs in the loop.

➤ A brief user guide about how to use our program:

- **Start the application:** Upon opening the application, a request to provide the start time will pop up. Hours, minutes, and seconds should be entered separately from 0 to 9.
- **Input Time:** The application will display something like "Input starting hours (0-9):". You will type in the number representing your intended starting time, and then you press Enter. Of course, you would do it similarly for the minute and second entries.
- **Display Clock:** Once the time is determined, the clock will start running, showing time in the format hh:mm:ss, refreshing automatically every second until it reaches the current time that was entered.

- **Stop Condition:** Once it reaches your entered time, it stops and says "Clock Stopped".
- **Stop Timer and End:** When the timer stops, it will end the program and return to the DOS command prompt.